

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з практичної роботи №3
з дисципліни: «Високорівневі мови програмування та фреймворки»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н.І.

Варіант: №13

Перевірив:

ст. викл.

Саманцов О. О.

3 ПРАКТИЧНА РОБОТА

3.1 Мета

Ознайомитися з основами мобільної розробки для платформи Android за допомогою мови програмування Kotlin. Набути практичних навичок роботи в середовищі Android Studio, опанувати базовий синтаксис Kotlin (типи даних, структури Map та CharArray, цикли, умовні конструкції), навчитися проєктувати графічні інтерфейси користувача за допомогою XML-розмітки, взаємодіяти з UI-елементами через код та реалізувати багатоекранну навігацію в межах одного додатка за допомогою компонентів Intent.

3.2 Індивідуальне завдання

Бажана оцінка: 100

Вимоги для отримання оцінки 100: одне завдання з кожного рівня та очний захист.

Номер в журналі 13.

1, 2, 3, 4 рівень завдання № 3 ($13 \% 10 = 3$)

Для виконання були отримані завдання:

- Рівень 1: Створіть форму з двома текстовими полями для вводу чисел. Реалізуйте кнопку, при натисканні якої сума введених чисел виводиться на екран.
- Рівень 2: Створіть додаток «Календар подій», де користувач може додавати та переглядати події за конкретну дату.
- Рівень 3: Створіть гру «поле чудес» (відгадування слів по літерам). можуть додавати завдання та присвоювати їм теги або мітки.
- Рівень 4: Реалізуйте конвертацію римських чисел в арабські.

3.3 Виконання завдання

3.3.1 Завдання 1

Реалізовано окремий екран CalcActivity з вертикальною розміткою LinearLayout. На екрані розміщено два поля введення типу EditText із обмеженням

на введення лише числових значень (numberDecimal), кнопку дій та текстове поле TextView для результату. У Kotlin-файлі налаштовано зчитування текстових потоків, додано валідацію на пусті поля (із виведенням попередження через Toast), виконано приведення типів до Double для підтримки дробових чисел, прораховано суму та виведено її на екран.

```
class CalcActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_calc)

        val number1 = findViewById<EditText>(R.id.number1)
        val number2 = findViewById<EditText>(R.id.number2)
        val calcButton = findViewById<Button>(R.id.calcButton)
        val resultText = findViewById<TextView>(R.id.resultText)
        val btnBack = findViewById<Button>(R.id.btnBack)

        calcButton.setOnClickListener {
            val str1 = number1.text.toString()
            val str2 = number2.text.toString()

            val d1 = str1.toDoubleOrNull()
            val d2 = str2.toDoubleOrNull()

            if (d1 == null || d2 == null) {
                Toast.makeText(this, "Please enter valid numbers",
                    Toast.LENGTH_SHORT).show()
            } else {
                val sum = d1 + d2
                resultText.text = "Result: $sum"
            }
        }

        btnBack.setOnClickListener {
            finish()
        }
    }
}
```

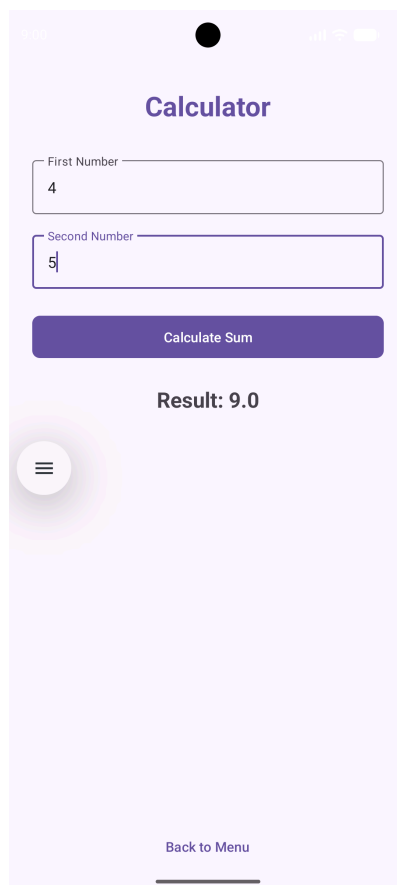


Рисунок 3.1 – Інтерфейс виконаного завдання 1

3.3.2 Завдання 2

Створено екран `CalendarActivity`. Для зручного вибору дати інтегровано виклик системного компонента `DatePickerDialog`, який синхронізується з поточним часом пристрою через клас `Calendar`. Для зберігання даних у пам'яті використано динамічну структуру `MutableMap` (асоціативний масив), де ключем є обрана дата у вигляді рядка, а значенням текст події. При виборі дати програма автоматично перевіряє карту на наявність записів і виводить або існуючу подію, або повідомлення про її відсутність. При натисканні кнопки «Save» дані валідуються та записуються в мапу.

```
class CalendarActivity : AppCompatActivity() {

    private var selectedDate = ""
    private val eventsMap = mutableMapOf<String, String>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_calendar)

        val selectDateButton =
```

```

findViewById<Button>(R.id.selectDateButton)
    val selectedDateText =
findViewById<TextView>(R.id.selectedDateText)
    val eventInput = findViewById<EditText>(R.id.eventInput)
    val saveEventButton =
findViewById<Button>(R.id.saveEventButton)
    val savedEventsDisplay =
findViewById<TextView>(R.id.savedEventsDisplay)
    val btnBack = findViewById<Button>(R.id.btnBack)

    selectDateButton.setOnClickListener {
        val calendar = Calendar.getInstance()
        val year = calendar.get(Calendar.YEAR)
        val month = calendar.get(Calendar.MONTH)
        val day = calendar.get(Calendar.DAY_OF_MONTH)

        val datePickerDialog = DatePickerDialog(this, { _,
selectedYear, selectedMonth, selectedDay ->
            selectedDate = "$selectedDay/${selectedMonth + 1}/
$selectedYear"
            selectedDateText.text = "Selected Date:
$selectedDate"

            val existingEvent = eventsMap[selectedDate]
            savedEventsDisplay.text = "Saved Events:
\n${existingEvent ?: "No events for this day"}"
            }, year, month, day)

        datePickerDialog.show()
    }

    saveEventButton.setOnClickListener {
        val eventDescription = eventInput.text.toString().trim()

        if (selectedDate.isEmpty()) {
            Toast.makeText(this, "Please select a date first",
Toast.LENGTH_SHORT).show()
            return@setOnClickListener
        }

        if (eventDescription.isEmpty()) {
            Toast.makeText(this, "Please enter event
description", Toast.LENGTH_SHORT).show()
            return@setOnClickListener
        }

        eventsMap[selectedDate] = eventDescription
        eventInput.text.clear()
        savedEventsDisplay.text = "Saved Events:\n$selectedDate:
$eventDescription"
        Toast.makeText(this, "Event saved successfully",
Toast.LENGTH_SHORT).show()
    }

    btnBack.setOnClickListener {
        finish()
    }
}
}

```

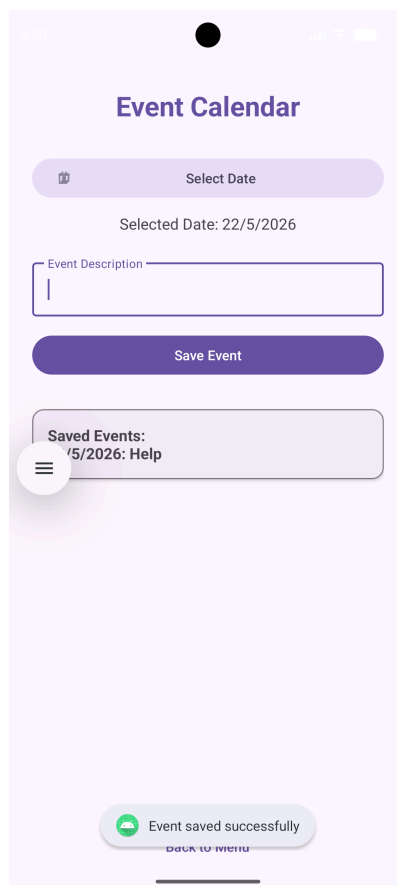


Рисунок 3.2 – Інтерфейс виконаного завдання 2

3.3.3 Завдання 3

Створено екран `GameActivity`. Загадане слово «KOTLIN» замасковано на екрані за допомогою масиву символів `CharArray`, заповненого зірочками. Реалізовано алгоритм перевірки: додаток зчитує літеру з поля `EditText` (з обмеженням довжини в 1 символ), переводить її в єдиний верхній регістр (`uppercase`) і через цикл `for` порівнює з кожною позицією в секретному слові. При збігу відкриваються відповідні літери, а при помилці зменшується лічильник спроб. Логіка відстежує фінальні стани (перемога/програш), виводить повідомлення та блокує кнопку гри.

```
class GameActivity : AppCompatActivity() {

    private val secretWord = "KOTLIN"
    private var displayedWord = CharArray(secretWord.length) { '*' }
    private var attemptsLeft = 6

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_game)
    }
}
```

```

val wordDisplay = findViewById<TextView>(R.id.wordDisplay)
val attemptsText = findViewById<TextView>(R.id.attemptsText)
val letterInput = findViewById<EditText>(R.id.letterInput)
val guessButton = findViewById<Button>(R.id.guessButton)
val btnBack = findViewById<Button>(R.id.btnBack)

wordDisplay.text = String(displayedWord)

guessButton.setOnClickListener {
    val input =
letterInput.text.toString().trim().uppercase()
    letterInput.text.clear()

    if (input.isEmpty()) {
        Toast.makeText(this, "Please enter a letter",
Toast.LENGTH_SHORT).show()
        return@setOnClickListener
    }

    val guessedLetter = input[0]
    var hit = false

    for (i in secretWord.indices) {
        if (secretWord[i] == guessedLetter) {
            displayedWord[i] = guessedLetter
            hit = true
        }
    }

    if (hit) {
        wordDisplay.text = String(displayedWord)
        if (!String(displayedWord).contains('*')) {
            Toast.makeText(this, "You won! Word is
$secretWord", Toast.LENGTH_LONG).show()
            guessButton.isEnabled = false
        }
    } else {
        attemptsLeft--
        attemptsText.text = "Attempts left: $attemptsLeft"
        if (attemptsLeft <= 0) {
            Toast.makeText(this, "Game Over! Word was
$secretWord", Toast.LENGTH_LONG).show()
            guessButton.isEnabled = false
        }
    }
}

btnBack.setOnClickListener {
    finish()
}
}
}

```

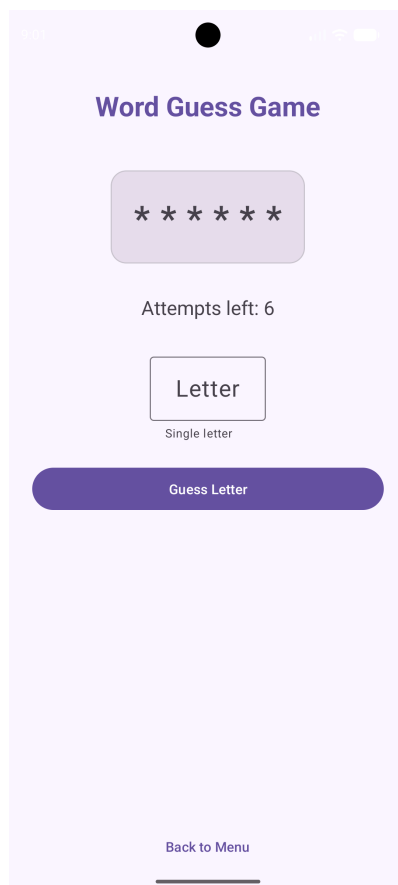


Рисунок 3.3 – Інтерфейс виконаного завдання 3 (до розгадки слова)

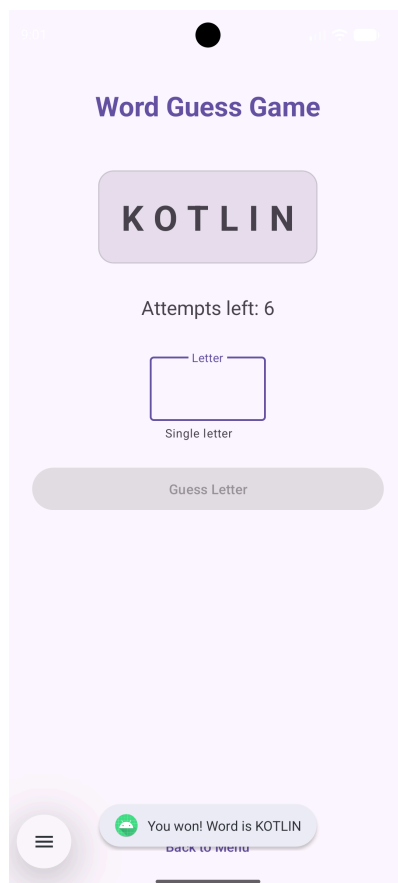


Рисунок 3.4 – Інтерфейс виконаного завдання 3 (після розгадки слова)

3.3.4 Завдання 4

Реалізовано екран ConverterActivity. Базовий математичний алгоритм використовує незмінну мапу mapOf для співвідношення римських символів (I, V, X тощо) з їхніми числовими еквівалентами. Рядок аналізується за допомогою циклу в зворотному напрямку (від останнього символу до першого). Якщо поточне значення символу менше за попереднє оброблене (наприклад, як I перед X у числі IX), воно віднімається від загального результату, інакше додається. Додано обробку помилок на випадок введення некоректних літер.

```
class ConverterActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_converter)

        val romanInput = findViewById<EditText>(R.id.romanInput)
        val convertButton = findViewById<Button>(R.id.convertButton)
        val arabicResultText =
            findViewById<TextView>(R.id.arabicResultText)
        val btnBack = findViewById<Button>(R.id.btnBack)

        convertButton.setOnClickListener {
            val roman = romanInput.text.toString().trim().uppercase()
            if (roman.isEmpty()) {
                Toast.makeText(this, "Please enter a Roman numeral",
                    Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            try {
                val result = romanToArabic(roman)
                arabicResultText.text = "Arabic Number: $result"
            } catch (e: Exception) {
                Toast.makeText(this, "Invalid Roman numeral",
                    Toast.LENGTH_SHORT).show()
            }
        }

        btnBack.setOnClickListener {
            finish()
        }
    }

    private fun romanToArabic(s: String): Int {
        val romanMap = mapOf('I' to 1, 'V' to 5, 'X' to 10, 'L' to
50, 'C' to 100, 'D' to 500, 'M' to 1000)
        var total = 0
        var prevValue = 0
        for (i in s.length - 1 downTo 0) {
            val currentValue = romanMap[s[i]] ?: throw
IllegalArgumentException("Invalid character")
            if (currentValue < prevValue) {
```

```

        total -= currentValue
    } else {
        total += currentValue
    }
    prevValue = currentValue
}
return total
}
}

```

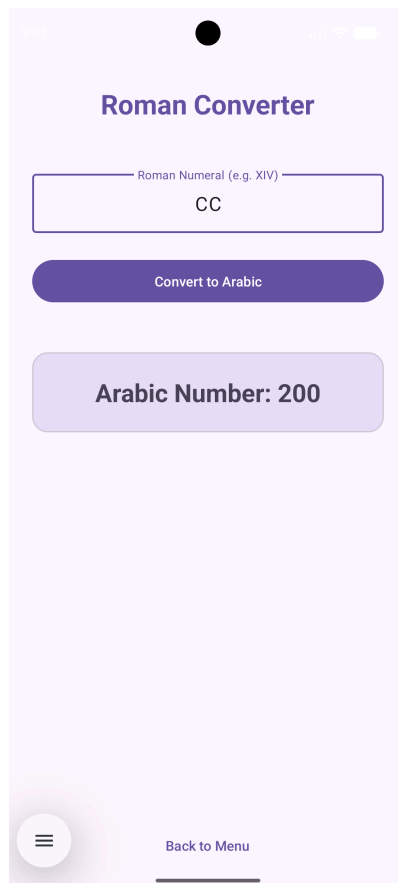


Рисунок 3.5 – Інтерфейс виконаного завдання 4

3.4 Висновки

У процесі виконання практичної роботи було опановано базові концепції мобільної розробки на платформі Android з використанням мови Kotlin. Створено чотири окремі екрани, кожен з яких реалізує різні функціональні можливості: від простого калькулятора до інтерактивної гри та конвертера римських чисел. Практика показала важливість правильного проектування інтерфейсу користувача, обробки подій та валідації даних для забезпечення коректної роботи додатка.

3.5 Контрольні запитання

1) Що таке Kotlin і які переваги він надає у порівнянні з іншими мовами програмування?

Kotlin — це сучасна об'єктно-орієнтована статично типізована мова програмування, що працює на базі Java Virtual Machine (JVM). Її головними перевагами є повна сумісність із Java, лаконічність коду (потребує на 40% менше рядків), вбудована безпека від нульових посилань (Null Safety) та підтримка сучасних концепцій, як-от корутини для асинхронної роботи.

2) Які базові типи даних підтримуються в Kotlin? Для кожного типу наведіть приклад.

В Kotlin підтримуються такі базові типи: цілі числа (Int, наприклад 42), числа з плаваючою комою (Double, наприклад 3.14), символи (Char, наприклад "A"), логічний тип (Boolean, наприклад true) та текстові рядки (String, наприклад «Kotlin»).

3) Що таке цикли в Kotlin? Наведіть приклади використання циклів.

Цикли — це керуючі конструкції для багаторазового виконання певної ділянки коду, поки виконується задана умова. В Kotlin використовуються цикл for (наприклад, для перебору діапазону: `for (i in 1..5) println(i)`) та цикли while / do-while (наприклад, `while (x > 0) { x-- }`).

4) Які типові структури даних використовуються в Kotlin? Поясніть, які це структури та дайте приклади.

Типовими структурами є масиви (Array, фіксований розмір: `arrayOf(1, 2, 3)`), списки (List, впорядкована послідовність: `listOf("A", "B")`), множини (Set, колекція унікальних елементів: `setOf(1, 2, 2)`) та асоціативні масиви (Map, пари ключ-значення: `mapOf("id" to 13)`). Колекції бувають незмінними (read-only) та змінними (MutableList, MutableMap).

5) Які конструкції розгалуження використовуються в Kotlin? Наведіть приклади використання кожної з них.

Використовується класичний умовний оператор if-else (наприклад: `if (x > 0) «Positive» else «Negative»`) та потужна багатоаconditionал конструкція when, яка замінює switch (наприклад: `when(x) { 1 -> println("One"); else -> println("Other") }`). Обидві конструкції в Kotlin можуть повертати значення як вирази.

6) Які особливості роботи з рядками в Kotlin? Наведіть приклади основних операцій з рядками.

Рядки в Kotlin представлені класом `String` і є незмінними. Особливостями є підтримка інтерполяції за допомогою знака `$` (наприклад: «Hi, \$name»), робота з багаторядковим текстом у потрібних лапках («»«...»«») та наявність зручних методів розбиття чи заміни елементів (наприклад: `str.substring(0, 2)` або `str.toUpperCase()`).

7) Яким чином можна працювати з файлами в Kotlin? Наведіть приклади читання, запису та видалення файлів.

Робота з файлами базується на стандартних класах Java, але розширена зручними Kotlin-методами `extension-функцій` через об'єкт `File`. Запис у файл: `File(«test.txt»).writeText(«Hello»);` читання вмісту: `val text = File(«test.txt»).readText();` видалення файлу з диска: `File(«test.txt»).delete()`.

8) Які методи та функції використовуються для роботи з файлами в Kotlin? Для кожного методу наведіть приклад його використання.

Для спрощення операцій використовуються функції `readText()` (зчитує весь файл у рядок), `readLines()` (повертає список рядків), `writeText()` (записує текст, перезаписуючи файл) та `appendText()` (дописує текст у кінець файлу). Приклад: `File(«log.txt»).appendText(«\nNew log entry»)`.

9) Як перевірити існування файлу в Kotlin? Наведіть приклад.

Перевірка існування файлу або директорії за вказаним шляхом виконується за допомогою вбудованого методу `exists()`, який повертає логічне значення `true` або `false`. Приклад: `if (File(«config.txt»).exists()) { }`.

10) Які основні функції використовуються для роботи з текстовими файлами в Kotlin? Для кожної функції наведіть приклад.

Основними функціями є `writeText(text)` для запису даних (приклад: `file.writeText(«data»)`), `readText()` для швидкого отримання всього вмісту (приклад: `val content = file.readText()`) та `forEachLine { ... }` для пострядкового зчитування великих файлів без перевантаження пам'яті.

11) Як зчитати вміст текстового файлу в Kotlin? Наведіть приклад.

Найпростіший спосіб зчитати весь вміст файлу в один рядковий об'єкт — викликати метод `readText()`.

```
val fileContent = java.io.File("input.txt").readText()
```

12) Як записати дані в текстовий файл в Kotlin? Наведіть приклад.

Запис текстових даних виконується методом `writeText()`, який автоматично створює файл, якщо його немає, або повністю перезаписує існуючий вміст.

```
java.io.File("output.txt").writeText("Sample data string")
```

13) Як видалити текстовий файл в Kotlin? Наведіть приклад.

Видалення файлу з файлової системи здійснюється за допомогою виклику методу `delete()`, який повертає `true` у разі успішного видалення.

```
isDeleted = java.io.File("cache.txt").delete()
```

14) Які класи використовуються для роботи з файлами в Android додатках на Kotlin?

В Android-розробці використовуються стандартні класи пакету `java.io` (такі як `File`, `FileInputStream`, `FileOutputStream`, `BufferedReader`) та класи контексту Android, зокрема `Context`, для безпечного доступу до внутрішніх папок додатка через методи `openFileInput()`, `openFileOutput()` або `filesDir`.

15) Як отримати доступ до файлів в Android додатках на Kotlin?

Для доступу до внутрішнього сховища додатка (`Internal Storage`), яке є приватним і безпечним, використовується контекст додатка. Запис файлу реалізується через `context.openFileOutput(«name.txt», Context.MODE_PRIVATE)`, а доступ до зовнішньої пам'яті (`External Storage`) вимагає додаткового декларування дозволів `READ_EXTERNAL_STORAGE` / `WRITE_EXTERNAL_STORAGE` у маніфесті програми.